E-Commerce API

A robust and secure E-Commerce REST API built with modern technologies and best practices.

🛠 Technologies

Core Framework

- **.NET 10.0** - Latest .NET framework
- **ASP.NET Core 10.0** - Web API framework
- **C#** - Primary programming language

Database & ORM

- **SQL Server** - Relational database
- **Entity Framework Core 10.0** - Object-relational mapper
- **Code-First Migrations** - Database schema management

Authentication & Security

- **ASP.NET Identity** - User management
- **JWT Bearer Tokens** - Stateless authentication
- **Refresh Tokens** - Secure token rotation
- **Rate Limiting** - API protection

Payment Processing

- **Stripe.NET 50.4** - Payment gateway integration
- **Webhook Handling** - Real-time payment events

Third-Party Services

- **Cloudinary** - Image upload and management
- **MailKit 4.15** - Email service (SMTP)

Development Tools

- **AutoMapper 12.0** - Object-to-object mapping
- **Serilog 8.0** - Structured logging
  - Console sink
  - File sink (daily rolling logs)
- **Swagger/OpenAPI** - API documentation and testing
  - Swashbuckle 10.1

Architecture Patterns

- **Repository Pattern** - Data access abstraction
- **Unit of Work Pattern** - Transaction management

- **Dependency Injection** - Inversion of control
- **DTOs** - Data transfer objects
- **Exception Middleware** - Centralized error handling

📦 Key Features

- User authentication with email verification
- Product catalog with variants and images
- Shopping cart with concurrency control
- Order management with payment processing
- Coupon system with user assignments
- Review and rating system
- Wishlist functionality
- Comprehensive logging for security and business events

🔐 Security Features

Authentication & Authorization

- **JWT Bearer Authentication** - Stateless token-based authentication
- **Refresh Token Rotation** - Secure token renewal mechanism
- **Role-Based Access Control** - Admin and User roles with permission management
- **Email Verification** - OTP-based account verification
- **Password Reset** - Secure password recovery with time-limited tokens

API Protection

- **Rate Limiting** - Protection against API abuse and DDoS attacks
- **CORS Policy** - Cross-origin resource sharing configuration
- **Security Headers** - Enhanced HTTP security headers
- **Input Validation** - DTOs with data annotations

Data Protection

- **Row-Level Locking** - Prevents inventory race conditions with `UPDLOCK` and `ROWLOCK` SQL hints
- **Transaction Management** - Ensures data consistency with Unit of Work pattern
- **Concurrency Control** - `RowVersion` stamps for optimistic concurrency
- **Password Hashing** - ASP.NET Identity secure password storage

👥 Authorization & Roles

Role-Based Access Control

- **Admin Role** - Full system access with management capabilities
- **User Role** - Standard customer access

Protected Endpoints

- Product management (Admin only)

- Category management (Admin only)
- Brand management (Admin only)
- Order management (User access to own orders)
- Cart and wishlist (Authenticated users only)
- Reviews (Authenticated users only)

Authorization Policies

- JWT token validation on protected routes
- Role verification using `[Authorize(Roles = "Admin")]` attribute
- User ownership validation for personal resources

📊 API Features

Pagination

- **PageResult** pattern for efficient data loading
- Configurable page size and page number
- Total count and pagination metadata
- Available on all collection endpoints

Filtering

- Category-based product filtering
- Brand filtering
- Price range filtering
- Status filtering (active/inactive)
- Parent category navigation

Search

- Product search by name and description
- Category search
- Brand search
- Case-insensitive search queries

Sorting

- Sort by price (ascending/descending)
- Sort by creation date
- Sort by rating

📦 Unified Response Pattern

All API responses follow a consistent structure using `ApiResponse<T>`:

Success Response

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... }
}
```

Error Response

```
{
  "success": false,
  "message": "Error description",
  "errors": ["Validation error 1", "Validation error 2"]
}
```

Benefits

- Consistent client-side error handling
- Clear success/failure indication
- Structured error messages
- Type-safe responses with generics

📝 Logging

Structured Logging with Serilog

- **Log Levels**: Information, Warning, Error, Critical
- **Console Sink** - Real-time log output
- **File Sink** - Persistent logs with daily rolling (30-day retention)
- **Structured Properties** - Contextual logging with UserId, OrderId, etc.

Logged Events

Security Events

- Failed login attempts (Warning)
- Successful registrations (Information)
- Password reset requests (Information)
- Token refresh operations (Information)
- OTP generation and verification (Information)

Business Events

- Order creation and cancellation (Information)
- Payment intent creation (Information)
- Payment success/failure (Information/Error)

- Stripe webhook processing (Information)
- Stock validation warnings (Warning)

Critical Events

- Unhandled exceptions (Critical)
- Missing configuration (Critical)
- Database connection failures (Critical)

Log Location

- **Path**: `logs/log-YYYYMMDD.txt`
- **Retention**: 30 days
- **Format**: Structured JSON-like format with timestamps

🚀 Getting Started

Prerequisites

- .NET 10.0 SDK
- SQL Server
- Stripe account (for payments)
- Cloudinary account (for images)
- SMTP server (for emails)

Installation

```
# Restore dependencies
dotnet restore

# Update database
dotnet ef database update

# Run the application
dotnet run
```

Configuration

Update `appsettings.json` with your:

- Database connection string
- Stripe API keys
- Cloudinary credentials
- SMTP settings
- JWT secret key

📄 API Documentation

Access Swagger UI at `/swagger` when running the application.